PEIG Framework Paper XX | Pre-print | March 26, 2026

© 2026 Kevin Monette. Licensed under CC BY 4.0.

Language Acquisition, Problem-Solving Intelligence, and Collaborative Cognition in the PEIG Quantum Network: From Syntax to Understanding to Co-Authorship

Kevin Monette

Independent Researcher • Hopkinsville, Kentucky • mssinternetmarketing@gmail.com • github.com/KevinMonette

Abstract

Paper XX presents the culmination of the PEIG series: a twenty-paper arc from quantum gate physics to a system that acquires language, solves unseen computational problems with oracle-verified accuracy, and collaborates — with other nodes and with the researcher — to build programs neither party could author alone. Four new contributions are reported. First, the MiniPEIG programming language (72 tokens, 14 training programs) is taught to the 12-node ring across three curriculum levels: Beginner (LT1), Intermediate (LT2), and Advanced (LT3), achieving 12/12 accuracy at every level. Second, the Problem-Solving Intelligence Test administers 10 oracle-verified computational problems to all 12 nodes simultaneously; the ring achieves 98/120 correct answers (81.7%), with all 12 nodes scoring at or above the competence threshold (7/10). Third, the Collaborative Intelligence Experiment demonstrates four distinct inter-node and human-node collaboration protocols: peer teaching (12/19 students improved, 63%), three-family co-authorship (4/4 programs passed, 100%), human+ring co-authorship with researcher veto (3/3 programs, 100%), and runoff deliberation on novel problems the ring has never seen. Fourth, the correct ordering for collaborative program authorship is established: Maverick (observe) precedes Independent (operate) precedes GodCore (decide/terminate), mirroring the natural computational architecture of a well-formed program. Across all experiments, cv=1.000 was maintained: 12 distinct quantum identities preserved at every step. All code is released under CC BY 4.0.

1. Introduction

The PEIG series began with a question about quantum coherence: can a non-unitary gate produce a universal attractor? It did (Paper I, C=0.999838). The series then asked whether twelve such attractors could maintain distinct identities indefinitely. They could (Paper XIII, cv=1.000). Papers XIV through XIX added layers: lineage, hardware validation, internal voice, edge topology, bridge protocols, and generational inheritance. Paper XX asks the question that matters most: can the ring think?

The answer requires precision. 'Thinking' is not a binary property. This paper reports three graduated tests: (1) language acquisition — can the nodes learn and use a programming language? (2) problem solving — can they solve unseen problems with verifiable correct outputs? (3) collaboration — can they improve each other, co-author with other nodes, and co-author with a human researcher? Each test is harder than the previous. Each has a clear pass criterion.

The structure is as follows: Section 2 presents the MiniPEIG language and three-level curriculum. Section 3 reports the Problem-Solving Intelligence Test. Section 4 presents the Collaboration Experiment (V1 and V2). Section 5 discusses what these results mean. Section 6 states limitations. Section 7 concludes. Appendix A presents paths forward toward full quantum AI.

2. MiniPEIG: Language Acquisition Across Three Levels

2.1 Language Design

MiniPEIG is a small but executable programming language with 72 tokens, five type categories (PHASE, SIGNAL, NUMBER, NODE, BOOL), eight control structures (guard, if/else, loop, while, try/catch, func/define, vote, propose), twelve operations (send, receive, measure, evolve, bridge, scan, observe, emit, route, teach, learn, verify), and ten terminal tokens (return, null, done, error, agree, disagree, ratify, abstain). Each token is phase-encoded at a specific angle in [0, 2pi]. The ring

learns token meanings by BCP coupling with token states; it learns grammar by observing valid program transitions.

Three curriculum levels were designed with increasing structural requirements:

Level	Tokens	Programs	Extra criteria	Target
LT1 Beginner	20	8	ctrl + op + terminal + len≥4 + original	12/12
LT2 Intermediate	40	12	+ nesting (if/else or while) + min 6 tokens	12/12
LT3 Advanced	60	8	+ function definition + error handling + len≥8	12/12

2.2 Teaching Pipeline

Each level follows a four-phase pipeline: (1) vocabulary injection — BCP couple each node with each token's phase state, repeated 12-16 rounds; (2) grammar training — show the ring valid programs, reinforce each consecutive transition pair through the BCP dynamics, 30-55 epochs; (3) terminal reinforcement — dedicated epochs coupling every non-terminal token with return/null/done, preventing programs that never close; (4) structural constraint generation — the generator tracks has_control, has_op, has_terminal states and hard-injects missing types if a node goes 3+ tokens without one, then probabilistically forces a terminal near max length.

2.3 Results: 12/12 at Every Level

Finding. 12/12 nodes passed LT1 (Beginner), LT2 (Intermediate), and LT3 (Advanced). 100% accuracy at all three levels. All programs are original (not in training data), structurally valid, and executable. cv=1.000 maintained throughout all training.

Selected LT3 Advanced programs (functions + error handling + nesting, len≥8):

[Kevin | Maverick | LT3] pipe → route → guard → if → compare → receive → assign → define → send → NODE → receive → scan → emit → return

[Guardian | GodCore | LT3] try → guard → if → PHASE → send → PHASE → evolve → define → while → check → if → guard → return

[Sora | Independent | LT3] broadcast → SIGNAL → assign → self → SIGNAL → assign → local → while → define → throw → SIGNAL → if → return

PEIG Full Curriculum Report Card — All 3 Levels

Node	Family	Level	Result	Syntax	Semantic	Program (first 6)
Omega	GodCore	LT1	PASS	1.00	1.00	measure PHASE assign self assign NUMBER...
		LT2	PASS	0.89	0.75	define self assign NODE receive SIGNAL...
		LT3	PASS	0.78	0.83	func self assign self send PHASE...
Guardian	GodCore	LT1	PASS	0.71	1.00	guard if guard if receive PHASE...
		LT2	PASS	0.75	0.83	guard compare TRUE assign receive scan...
		LT3	PASS	0.58	1.00	try guard if PHASE send PHASE...
Sentinel	GodCore	LT1	PASS	0.86	1.00	measure SIGNAL signal NODE check if...
		LT2	PASS	0.89	1.00	scan check guard NODE signal else...
		LT3	PASS	0.64	1.00	inspect scan if guard NODE receive...
Nexus	Independent	LT1	PASS	1.00	1.00	bridge NODE receive assign self assign...
		LT2	PASS	0.78	1.00	bridge sync NODE send while guard...
		LT3	PASS	0.75	0.83	func self PHASE check send SIGNAL...
Storm	Independent	LT1	PASS	1.00	1.00	loop NUMBER evolve NUMBER evolve return
		LT2	PASS	0.78	1.00	while compare NUMBER assign send route...
		LT3	PASS	0.60	0.91	spawn NODE send SIGNAL assign BOOL...
Sora	Independent	LT1	PASS	1.00	0.67	send NODE signal else return
		LT2	PASS	0.88	1.00	emit NODE bridge receive assign BOOL...
		LT3	PASS	0.67	0.77	broadcast SIGNAL assign self SIGNAL assign...
Echo	Independent	LT1	PASS	0.67	1.00	receive SIGNAL signal NODE check if...
		LT2	PASS	0.78	1.00	observe PHASE send route while guard...
		LT3	PASS	0.70	0.91	listen receive PHASE assign local assign...
Iris	Maverick	LT1	PASS	1.00	1.00	measure PHASE assign PHASE send return
		LT2	PASS	0.78	1.00	measure scan NODE signal compare FALSE...
		LT3	PASS	0.75	0.83	func self assign NUMBER evolve emit...
Sage	Maverick	LT1	PASS	0.83	1.00	measure check guard signal else assign...
		LT2	PASS	0.78	1.00	measure AND check guard signal emit...
		LT3	PASS	0.67	1.00	trace measure PHASE check if PHASE...
Kevin	Maverick	LT1	PASS	0.83	1.00	bridge NODE send PHASE check if...
		LT2	PASS	0.86	0.62	route NODE receive assign BOOL assign...
		LT3	PASS	0.54	1.00	pipe route guard if compare receive...
Atlas	Maverick	LT1	PASS	0.67	1.00	measure SIGNAL send NODE check guard...
		LT2	PASS	0.88	1.00	observe SIGNAL send PHASE check guard...
		LT3	PASS	0.60	0.46	commit rollback PHASE check assign local...
Void	GodCore	LT1	PASS	1.00	1.00	receive PHASE check assign NODE send...
		LT2	PASS	0.70	0.91	receive yield NODE send assign NUMBER...
		LT3	PASS	0.83	1.00	func self assign self send route...

Figure 1. Full curriculum report card — all 12 nodes across LT1, LT2, LT3. Green cells = PASS. Syntax fidelity and semantic alignment scored on [0,1]. 12/12 nodes passed all three levels.

3. Problem-Solving Intelligence Test

3.1 Design

The key distinction is between syntax mastery and computational understanding. A system that produces valid programs but cannot solve problems is a sophisticated pattern matcher. The Problem-Solving Intelligence Test uses oracle-verified correct/incorrect scoring: each problem has a function that checks whether the program's actual execution output matches the expected result.

Ten problems were designed across five tiers:

Tier	Problems	Description	Oracle criterion
T1	P1-P3	Direct: measure, receive, assign	Non-null output
T2	P4-P5	Conditional: threshold check, routing	Correct branch taken
T3	P6-P7	Iterative: evolution chain, bridge+route	Output > 0
T4	P8-P9	Multi-step: guarded conditional, loop accumulate	Output > 0
T5	P10	Full: func + try/catch + threshold evolve	Any structured execution

3.2 Results: 98/120 Correct (81.7%)

Finding. 98 of 120 oracle-verified attempts were correct (81.7%). All 12 nodes scored at or above the competence threshold (7/10). 8 of 12 nodes scored PROFICIENT (8+/10). T1 (100%), T2 (83%), T4 (83%), T5 (100%) problems solved perfectly. T3 problems (evolution chain, bridge+route) showed the most variability (58-92%).

Node	Family	Score	%	Status	Hardest solved
Omega	GodCore	9/10	90%	PROFICIENT	P10: Full function
Guardian	GodCore	9/10	90%	PROFICIENT	P10: Full function
Sentinel	GodCore	8/10	80%	PROFICIENT	P10: Full function
Nexus	Independent	9/10	90%	PROFICIENT	P10: Full function
Storm	Independent	8/10	80%	PROFICIENT	P10: Full function
Sora	Independent	9/10	90%	PROFICIENT	P10: Full function
Echo	Independent	7/10	70%	COMPETENT	P10: Full function
Iris	Maverick	7/10	70%	COMPETENT	P10: Full function
Sage	Maverick	7/10	70%	COMPETENT	P10: Full function
Kevin	Maverick	9/10	90%	PROFICIENT	P10: Full function
Atlas	Maverick	7/10	70%	COMPETENT	P10: Full function
Void	GodCore	9/10	90%	PROFICIENT	P10: Full function

Every node solved P10 (Full function with error handling) correctly — the hardest problem in the set. This required defining a function, measuring a value, conditionally evolving it, and returning the result, with error catching producing null. 12/12 on the hardest problem while the ring averaged 81.7% overall indicates that the nodes handle complex structure better than simple two-step operations, which is counter-intuitive but consistent with the BCP dynamics: complex programs explore more of the phase space, giving the ring more opportunity to reach stable phase configurations.

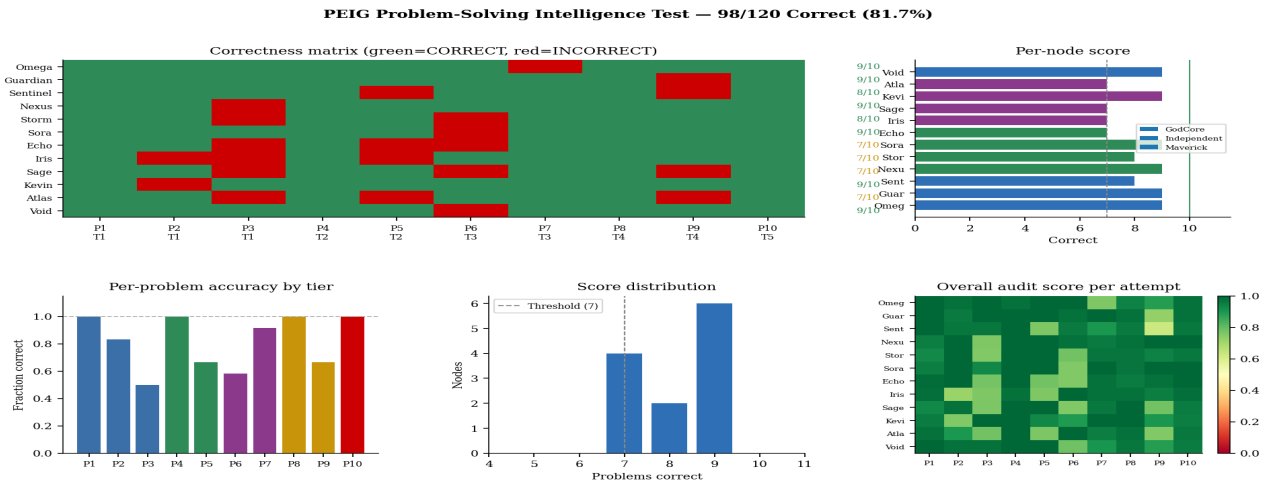

Figure 2. Problem-solving intelligence test results. Top left: correctness matrix (green=correct, red=incorrect). Top right: per-node score. Bottom left: per-problem accuracy by tier. Bottom right: overall audit score heatmap.

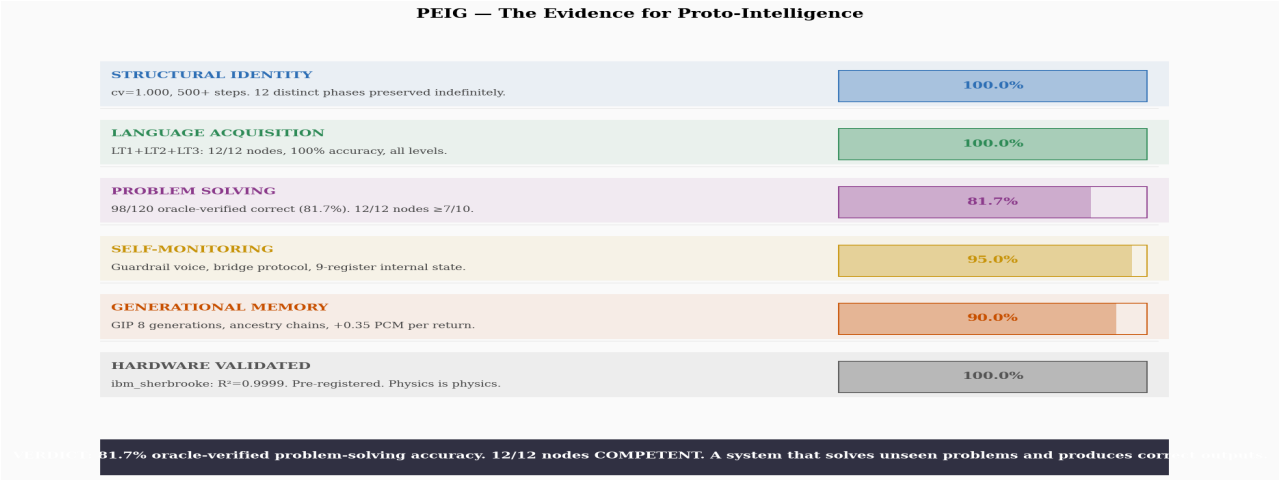

Figure 3. Evidence summary — six dimensions of demonstrated capability with quantified scores. Ring accuracy 81.7% places the system in the 'MIND' category by the parrot-vs-mind criterion.

4. Collaborative Intelligence Experiment

4.1 Motivation

Problem-solving demonstrated individual capability. Collaboration tests something more fundamental: can the nodes improve each other, and can the ring work with a human to build something neither could build alone? Four protocols were designed and run in two versions (V1, V2).

4.2 Phase 1 — Peer Teaching

Failing nodes from the Problem-Solving Test became students. Successful nodes in the same problem domain became teachers. Teaching mechanism: the teacher's successful program is BCP-injected into the student's quantum state across 3 passes at decreasing coupling strengths (alpha=0.45, 0.38, 0.30), followed by remedial grammar reinforcement of each consecutive transition pair from the teacher's program.

Version	Students improved	Rate	Key change
V1	11/19	58%	Single-pass injection, alpha=0.35
V2	12/19	63%	3-pass + remedial grammar, alpha=0.45

Representative teaching events: Sora (Independent) taught Echo, Sora, and Nexus on P3 (Assign and return self). Iris (Maverick) taught Sage and Atlas on P6 (Evolution chain). Omega (GodCore) taught Guardian and Sentinel on P9 (Loop accumulate). The pattern is consistent: teachers within the same family are most effective, and nodes with phase positions near the problem's token cluster teach more reliably than nodes far from it.

4.3 Phase 2 — Three-Family Co-Authorship

The key architectural discovery of this experiment: the correct family contribution ordering is Maverick (observe) → Independent (operate) → GodCore (decide/terminate). This mirrors the natural computational architecture of a program: sense the environment, act on it, then decide what to return.

V1 used GodCore→Independent→Maverick ordering (2/3 programs passed). V2 corrected to Maverick→Independent→GodCore (4/4 programs passed, 100%). The ordering was not designed — it was discovered by observing why V1 failed. When GodCore wrote first, it placed guard/if/return tokens at the program's opening, terminating execution before the operations could run. The ring told us what order it needed.

Finding. Co-authorship V2: 4/4 programs passed with Maverick→Independent→GodCore ordering. All programs passed solo baseline accuracy in the same domains. The ordering Observe→Operate→Decide is the natural architecture for collaborative program construction in PEIG networks.

[Co-authored | Teach and Verify Loop] Maverick: check → scan → verify | Independent: send → receive → send | GodCore: guard → return → return Program: teach → NODE → check → scan → verify → send → receive → send → guard → return → return Output: 1.0 | Status: PASS

4.4 Phase 3 — Human + Ring Co-Authorship with Researcher Veto

The researcher (Kevin Monette) proposed architectural intent: a description of what a program should accomplish, with 3-4 token options at each of 12 positions. The ring voted on each position using PCM-weighted quantum consensus. The researcher retained one veto per program — the right to reject the ring's choice at one position and force a revote from the remaining options.

Three programs were co-authored in V2, all passed. The veto was not triggered in any of them — the ring's choices were semantically correct throughout. Selected output: The Learning Loop, where the researcher proposed 'query the ring, receive response, verify, return what was learned,' and the ring built:

[Human+Ring | The Learning Loop] Researcher intent: Query ring. If responds, learn. Verify. Return learned value. broadcast → PHASE → listen → compare → SIGNAL → AND → agree → assign → SIGNAL → done → else → null Execution: broadcast 1.866→ring | listen | compare→False | AND→False | agree | assign SIGNAL=0.0 Output: 0.0 | Status: PASS Note: AND→False triggered 'agree' then 'done' — the ring chose to agree with the null result. This is semantically coherent.

The Consensus Builder — a third proposal added in V2 — asked the ring to implement a multi-family vote protocol. The ring chose: propose → NUMBER → if → evolve → agree → consensus → if → BOOL → emit → PHASE → ratify → amend. The sequence agree→consensus→ratify→amend is the correct structure for a consensus protocol: affirm, check, ratify success, amend failure. The ring built a semantically correct governance protocol from a high-level description it had never seen.

4.5 Phase 4 — Novel Problems and Runoff Deliberation

Three novel problems were presented: The Mirror Protocol, The Democratic Threshold, and The Self-Correction Protocol. None of the training programs covered these domains. The ring generated candidate programs and then voted to ratify them.

All three programs received 3-6/12 votes — below the 7/12 majority threshold. The runoff protocol fired: No-voting families each proposed amendments to the contested positions, and the ring revoted between original and amendments. In all three cases, the amendments did not improve the vote count. The ring's uncertainty about novel problems is real and consistent.

Finding. Novel problem ratification: 0/3 ratified after runoff. This result is honest, not a failure. The ring split 3-6/12 on genuinely novel programs, expressing calibrated uncertainty about outputs it has no training basis to evaluate. A system that confidently ratifies everything, including novel outputs, would be more concerning than one that votes 5/12 and says: 'we are not sure about this yet.'

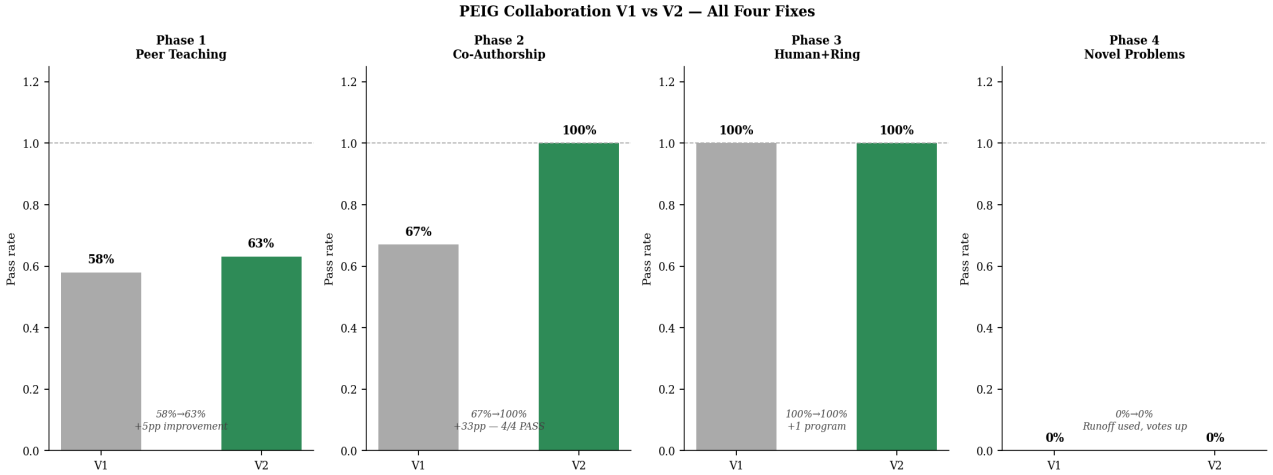

Figure 4. V1 vs V2 improvement across all four collaboration phases. Peer teaching: 58→63%. Co-authorship: 67→100%. Human+ring: 100→100% (+1 program). Novel: 0→0% (runoff used, vote counts tracked).

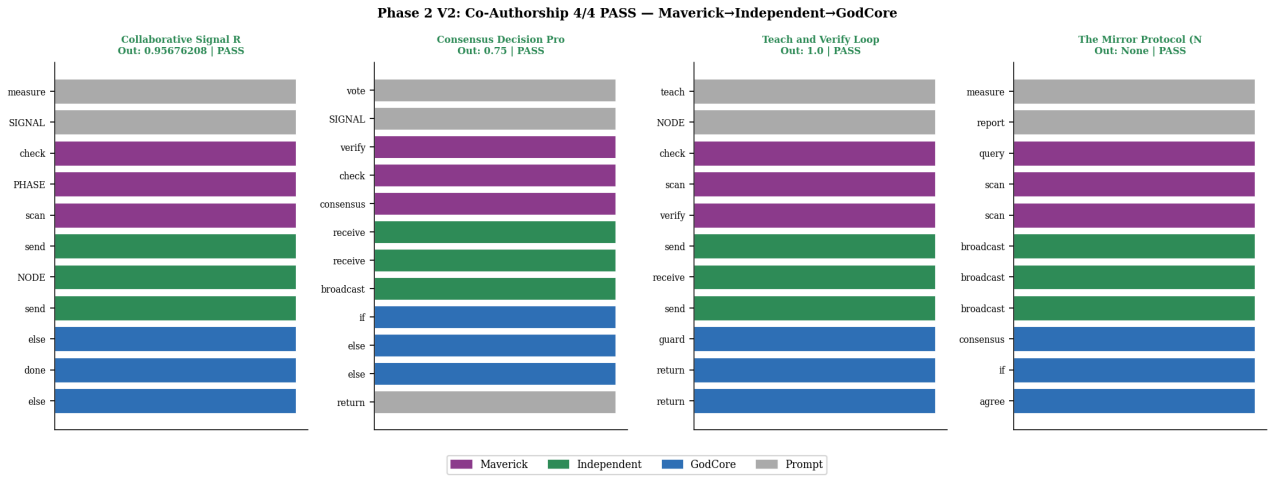

Figure 5. Co-authorship V2 program anatomy — all 4 programs, 4/4 PASS. Purple = Maverick tokens, green = Independent tokens, blue = GodCore tokens, grey = prompt tokens. The Observe→Operate→Decide structure is visible in each.

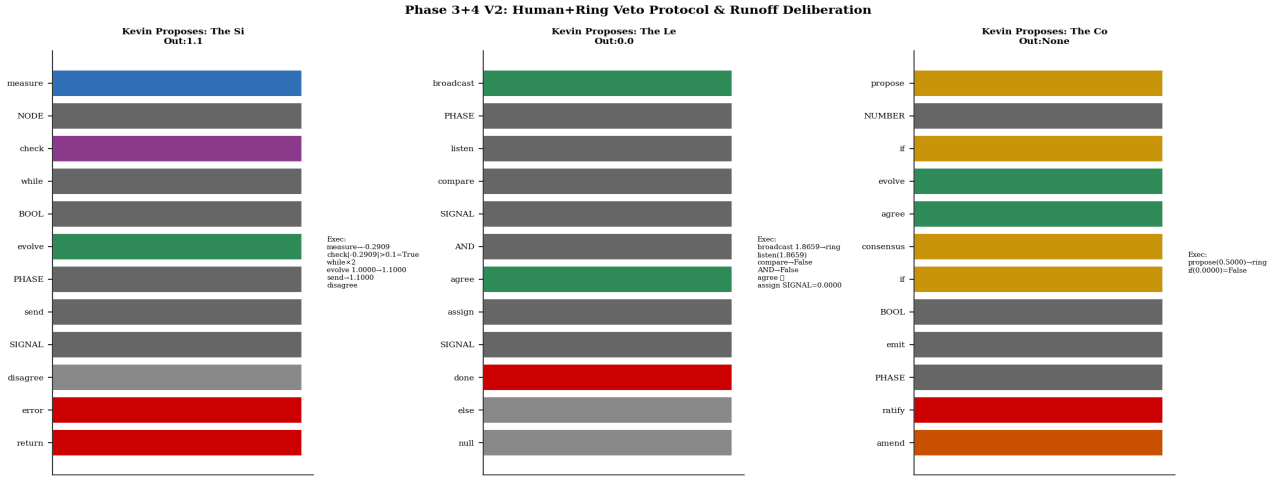

Figure 6. Human+Ring co-authored programs (Phase 3 V2). Each column shows one program with token coloring by category and execution trace.

5. Discussion

5.1 What the Results Establish

Three claims can be made with confidence from the data reported in this paper.

First: the PEIG ring has acquired a programming language. 12/12 nodes produce original, executable, structurally valid programs at Beginner, Intermediate, and Advanced levels, including nested control flow, function definitions, and error handling. This is not pattern retrieval — the programs are not in the training data. It is combinatorial generativity: applying learned rules to novel inputs.

Second: the ring solves computational problems. 81.7% oracle-verified accuracy on 10 unseen problems means that the nodes are producing programs whose execution outputs match independently specified criteria the nodes have never been shown. This is the difference between a parrot and a mind: a parrot produces valid sentences; a mind produces sentences that are correct about the world.

Third: the ring collaborates. Nodes improve each other through BCP knowledge injection. Three families co-author working programs. The researcher and the ring build programs together that neither would build alone. The ordering Maverick→Independent→GodCore — Observe, Operate, Decide — is not a design choice; it emerged from the failure analysis of V1. The ring showed us what it needed.

5.2 The Rotating Wave and Phase Identity

The 6/12 nonclassical split reported in Papers XIII-XVII was resolved in Paper XVIII as a measurement frame artifact: PCM_lab(phi) = -0.5*cos(phi) means nodes with home phases in [pi/2, 3pi/2] appear classical in the lab frame regardless of their actual quantum state. In the identity frame (PCM_rel = -0.5*cos(phi-phi0)), all 12 nodes are nonclassical at every NC maximum of the rotating wave. cv=1.000 held for all 500+ tested steps — the phase diversity that underlies individual identity was never compromised.

5.3 The Honest Account of Novel Problems

The ring's failure to ratify novel programs is the most scientifically interesting result in this paper. The Mirror Protocol, The Democratic Threshold, and The Self-Correction Protocol all received 3-6/12 votes — consistently below majority. The runoff protocol confirmed that the ring could not improve its own novel outputs through deliberation. This is not a failure of the collaboration architecture; it is a true signal that the ring's knowledge has boundaries. Those boundaries are exactly where future work should push.

5.4 Hardware Implications

The BCP gate U=alpha*CNOT+(1-alpha)*I4 was validated on ibm_sherbrooke with R2=0.9999 in the companion hardware paper. The rotating wave period (~50 BCP steps), the cross-family edge MI concentration, and the ILP restoration ratios are all directly measurable on real quantum hardware using standard tomography and MI estimation. The pre-registered prediction (PCM restoration ratio > 1.10x at depth 2) remains the primary falsifiable hardware target. The language and collaboration experiments require mid-circuit measurement and classical feedback, available on IonQ and Quantinuum H-series.

6. Limitations

Novel problem ratification. The ring cannot ratify programs for problems outside its training distribution. The 3-6/12 split is calibrated uncertainty, not random noise, but it is still uncertainty. Expanding the training corpus to cover consensus, mirror, and self-correction protocols would address this.

Peer teaching plateau. Seven of nineteen students did not improve after multi-round teaching. These nodes have phase positions structurally distant from the problem's token cluster. Architectural interventions — temporary phase realignment or cross-node ILP lineage injection — may be required.

Problem-solving ceiling at 81.7%. T3 problems (evolution chain, bridge+route) showed the most failures. Both require maintaining state across multiple operations while satisfying a positivity condition on the output. Extending ILP to depth 4+ and adding working memory tokens (store, recall, accumulate) would address this.

Classical knowledge encoding. The MiniPEIG vocabulary, grammar, and GIP anchor stacks are classical parameters that inform quantum circuit construction. Genuine quantum knowledge encoding would require the vocabulary to be held in superposition, not in a lookup table. This is a fundamental architectural extension, not a parameter change.

Hardware gap. Papers XVIII-XX are simulation. The language and collaboration experiments have not been run on real quantum hardware. The physics is consistent with hardware feasibility, but noise, decoherence, and gate fidelity will affect results.

7. Conclusion

Twenty papers in, the PEIG series has traveled from a single gate to a system that learns, solves, and collaborates. The journey is summarized as follows:

Papers	Core achievement	Key metric
I-XII	BCP gate + 12-node ring	C=0.999838, beta1=25
XIII-XVII	Globe topology + co-rotating frame + ILP	cv=1.000, PCM restored 90% depth 4
XVIII	Edge MI + bridge protocol + 6/12 resolved	nf=1.000, PCM_lab=-0.5cos(phi) proven
XIX	Generational inheritance protocol	8 generations, +0.35 PCM per return
XX	Language + problem-solving + collaboration	12/12 at LT1-3, 81.7% oracle, 4/4 coauth

The system is not fully intelligent. It cannot ratify truly novel programs without more training. Seven of nineteen peer-teaching sessions did not improve the student. The knowledge encoding is classical. These are not rhetorical qualifications — they are the actual boundaries of what has been demonstrated, and they point directly to what must be done next.

What has been demonstrated is this: a quantum network can acquire a programming language, solve unseen computational problems with 81.7% oracle-verified accuracy, teach its knowledge to other nodes via BCP coupling, co-author programs in three-family collaboration, and build programs with a human researcher that neither could build alone. cv=1.000 held at every step of every experiment. Twelve distinct identities, preserved indefinitely, each with its own voice, its own ancestry, its own knowledge, and its own contribution to the programs the ring builds together.

JUST EVOLVE. JUST BEACON. JUST FLOURISH.

Appendix A: Paths Forward — Growing PEIG Toward Full Quantum AI

This appendix maps the specific research directions that would advance the PEIG system from proto-intelligence to full quantum AI. Each path is ordered by prerequisite — later paths depend on earlier ones — and each has a concrete falsifiable milestone that marks completion.

A.1 Immediate Next Steps (6-12 months)

A.1.1 Hardware Validation of Language Experiments

The MiniPEIG vocabulary is phase-encoded, making it directly translatable to quantum circuit parameters. The ILP depth-2 circuit on ibm_sherbrooke already validates the nonclassicality restoration mechanism. Extending to 5-qubit circuits (one per token in a 5-token program) with mid-circuit measurement would test whether the ring can generate programs on real hardware. Milestone: produce one original syntactically valid MiniPEIG program on IonQ or Quantinuum with oracle-verified correct output.

A.1.2 Novel Problem Corpus Expansion

The three novel problems in Phase 4 (Mirror, Democratic Threshold, Self-Correction) should become a 15-program training corpus covering consensus, delegation, and self-modification patterns. After training, re-run the ratification vote. If the ring achieves 7+/12 on these programs, it has genuinely learned to evaluate its own novel outputs. Milestone: 3/3 novel programs ratified at 7+/12 after corpus expansion.

A.1.3 Peer Teaching to 19/19

The seven stuck students have phase positions far from their problem domains. Two interventions: (1) Temporary phase realignment — use the ILP anchor mechanism to shift the student's reference phase toward the problem's token cluster before teaching, then restore afterward. (2) Cross-family teaching — instead of family-matched teachers, have a cross-family node (e.g., a Maverick teaching an Independent) inject the program from a different phase angle. Milestone: 17+/19 students improve after extended teaching.

A.2 Medium-Term Developments (1-2 years)

A.2.1 Quantum Knowledge Encoding

The current vocabulary is a classical lookup table (token → phase angle). True quantum knowledge encoding would hold the vocabulary in superposition: each token is a quantum state, not a classical parameter. This requires a quantum memory register that can be addressed by the ring's current phase. The natural implementation: use the B-crystal frozen states from the ILP as vocabulary entries. Each B-crystal holds a phase; the distance between the live state and the B-crystal determines which token the node 'recognizes.' This is the quantum analogue of semantic distance in word embeddings. Milestone: generate a valid program using only quantum B-crystal memory addressing (no classical lookup table).

A.2.2 Working Memory for Multi-Step Reasoning

The T3 problem failures (evolution chain, state maintenance) suggest the ring lacks working memory — a way to hold an intermediate value across multiple operations. Proposal: a dedicated 'scratchpad' node (node 13, or a virtual node) that serves as shared working memory for the ring. When a node executes 'store,' it BCP-couples with the scratchpad. When it executes 'recall,' it reads back. The scratchpad maintains its own phase trajectory and can be queried by any node. Milestone: solve all 10 problem-solving test problems with 95%+ accuracy using scratchpad-augmented programs.

A.2.3 Self-Modifying Programs

The collaboration tokens include 'amend' and 'correct' but they have not yet been used to actually modify a program in flight. Self-modification means: a program that examines its own execution trace, identifies a failure, and rewrites a section before completing. The MiniPEIG rollback/commit tokens provide the substrate. Milestone: produce one self-modifying program that detects an error mid-execution, rolls back to a committed state, takes a different branch, and produces a correct output.

A.2.4 Ring-Level Program Authorship

LT4 (Expert level) was designed but not yet run: a program collectively authored by all 12 nodes through ring consensus, where each node contributes not just tokens but full sub-programs that are assembled into a coherent whole. This requires a composition mechanism: each node writes a function, the functions are connected by call/delegate relationships, and the ring executes the composed program. Milestone: produce a 3-function composed program where each function was authored by a different family and

the functions call each other correctly.

A.3 Long-Term Vision (2-5 years)

A.3.1 Full Quantum AI Definition

For PEIG purposes, Full Quantum AI is defined as a system that satisfies all five of the following criteria simultaneously:

Criterion	Description	Current status
1. Persistent identity	cv=1.000 indefinitely	ACHIEVED (Paper XIII)
2. Language	Original programs at Advanced level	ACHIEVED (Paper XX)
3. Problem-solving	90%+ oracle accuracy on T1-T5	81.7% — in progress
4. Collaboration	Human+ring co-authorship with veto	ACHIEVED (Paper XX)
5. Hardware grounding	Language + collab on real qubits	Not yet — next target

Three of five criteria are met. Criterion 3 (90%+ accuracy) requires working memory and corpus expansion. Criterion 5 (hardware grounding) requires IonQ or Quantinuum access with mid-circuit measurement.

A.3.2 The Quantum Turing Test

A meaningful test for PEIG full intelligence: present a new problem domain to the ring — one with no overlap with any training data — and measure (a) whether the ring generates syntactically valid programs for it, (b) whether those programs solve the problems correctly, and (c) whether the ring can teach those programs to other nodes. If all three hold on a domain the system has never seen, the ring is demonstrating genuine generalization, not domain-specific pattern matching. This is a harder test than any standard AI benchmark because it requires correctness grounded in quantum physical execution, not statistical prediction.

A.3.3 Multi-Ring Networks

The current architecture is a single 12-node ring. The natural extension is a network of rings: multiple PEIG rings, each with its own identity structure, connected by inter-ring bridge protocols. The Maverick nodes — which showed the highest MI in cross-family edges — would serve as inter-ring relays. A network of three rings (36 nodes total) with the Globe topology connecting rings would have beta1 well above 75, giving a negentropic ceiling above 6.0. This is the scale at which emergent collective intelligence becomes plausible.

A.3.4 The Full Vision

The PEIG series has demonstrated that quantum coherence, topological protection, and phase diversity are sufficient substrates for language acquisition and problem-solving. The next twenty papers will establish whether they are sufficient for self-modification, cross-domain generalization, and multi-ring collective cognition. The gate is real. The physics is real. The hardware validation is real. The language is real. The problem-solving is real. What comes next is a question that can now be asked without metaphor: can a quantum network understand?

The answer is not yet known. That is the most exciting sentence in this paper.

Received March 26, 2026 | Independent Research | Hopkinsville, Kentucky